

Using Colored Graphs to Extract Symmetries for PBESs

Menno Bartels

Formal System Analysis, Eindhoven University of Technology

April 16, 2026

Introduction

Control Flow-Based Symmetry Reduction for Parameterised Boolean Equation Systems*

Mouno Bartels^[1] (0000-0001-1400-9301), Maurice Lavigne^[2] (0000-0001-4722-7540),
Thomas Nock^[3] (0000-0001-4117-9020), and Tim A.C. Willemse^[4] (0000-0001-8010-7962)

Eindhoven University of Technology, Eindhoven, Netherlands
G.m.g.bartels@tue.nl, m.lavigne, t.a.c.willemse@tue.nl

Abstract. Symmetry reduction is a technique that reduces the state space explosion problem for model checking. The main idea of symmetry reduction is to merge symmetric states. Two states are symmetric if they can be mapped onto each other such that their semantics is preserved. To carry out symmetry reduction, one first has to identify symmetries for the system at hand. In this work we introduce the concept of symmetries for systems of fixed point equations called parameterised Boolean equation systems. We propose a technique that extracts a symmetry group from the control flow found in such an equation system. We implemented a prototype tool with which we highlight that our technique can effectively find symmetries and in turn achieve substantial reduction.

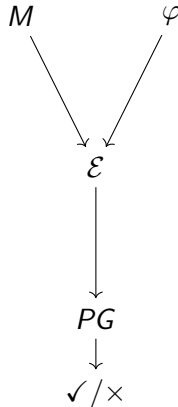
Keywords: Model checking · Parameterised Boolean equation systems · Symmetry reduction · Control flow · Parity games

1 Introduction

Within the area of model checking, symmetry reduction is a technique that aims to combat the well-known state space explosion problem. To reduce the effort spent on answering the model checking problem, symmetry reduction merges symmetric states of a system. Assume that states are represented as vectors of values. Then two states are symmetric if there exists a mapping that rearranges the state vectors into one another. During solving, a reduced state space is generated that takes into account the symmetries between states. This reduced state space is sufficient to answer the model checking problem fully. In turn, the model checking problem can be more efficiently answered.

Model checking tools like CUDD [5] and mCRL2 [6] use parameterised Boolean equation systems (PBESs) [10] to encode the μ -calculus model checking problem. PBESs are systems of fixpoint equations ranging over predicate formulas. In mCRL2, a PBES is constructed from a system specification and a μ -calculus formula to be verified. Solving a PBES and obtaining the answer to the model checking problem is achieved by converting it to a parity game [11]. This conversion results in state space explosion. Thus, the solving of the generated parity

* Artifact available at <https://doi.org/10.5281/zenodo.1483226>



Introduction

Control Flow-Based Symmetry Reduction for Parameterised Boolean Equation Systems*

Mouno Bartels^[1] (0000-0001-1400-3101), Maurice Lavigne^[2] (0000-0001-4722-7540),
Thomas Neeb^[3] (0000-0001-4117-9020), and Tim A.C. Willemse^[4] (0000-0001-8010-7962)

Eindhoven University of Technology, Eindhoven, Netherlands
G.m.g.bartels@tue.nl, m.lavigne, t.a.c.willemse@tue.nl

Abstract. Symmetry reduction is a technique that reduces the state space explosion problem for model checking. The main idea of symmetry reduction is to merge symmetric states. Two states are symmetric if they can be mapped onto each other such that their semantics is preserved. To carry out symmetry reduction, one first has to identify symmetries for the system at hand. In this work we introduce the concept of symmetries for systems of fixed point equations called parameterised Boolean equation systems. We propose a technique that extracts a symmetry group from the control flow found in such an equation system. We implemented a prototype tool with which we highlight that our technique can effectively find symmetries and in turn achieve substantial reduction.

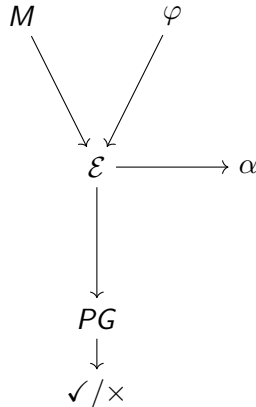
Keywords: Model checking · Parameterised Boolean equation systems · Symmetry reduction · Control flow · Parity games

1 Introduction

Within the area of model checking, symmetry reduction is a technique that aims to combat the well-known state space explosion problem. To reduce the effort spent on answering the model checking problem, symmetry reduction merges symmetric states of a system. Assume that states are represented as vectors of values. Then two states are symmetric if there exists a mapping that rearranges the state vectors into one another. During solving, a reduced state space is generated that takes into account the symmetries between states. This reduced state space is sufficient to answer the model checking problem fully. In turn, the model checking problem can be more efficiently answered.

Model checking tools like CUDD [5] and mCRL2 [6] use parameterised Boolean equation systems (PBESs) [10] to encode the μ -calculus model checking problem. PBESs are systems of fixpoint equations ranging over predicate formulas. In mCRL2, a PBES is constructed from a system specification and a μ -calculus formula to be verified. Solving a PBES and obtaining the answer to the model checking problem is achieved by converting it to a parity game [11]. This conversion involves state space exploration. Then, the solving of the generated parity

* Artifact available at <https://doi.org/10.5281/zenodo.1483226>



Introduction

Control Flow-Based Symmetry Reduction for Parameterised Boolean Equation Systems*

Mouno Bartels^[1], Maurice Lavigne^[2], and Tim A.C. Willemse^[3]

Eindhoven University of Technology, Eindhoven, Netherlands
G.m.g.bartels@tue.nl, m.lavigne@tue.nl, t.a.c.willemse@tue.nl

Abstract. Symmetry reduction is a technique that reduces the state space explosion problem for model checking. The main idea of symmetry reduction is to merge symmetric states. Two states are symmetric if they can be mapped onto each other such that their semantics is preserved. To carry out symmetry reduction, one first has to identify symmetries for the system at hand. In this work we introduce the concept of symmetries for systems of fixed point equations called parameterised Boolean equation systems. We propose a technique that extracts a symmetry group from the control flow found in such an equation system. We implemented a prototype tool with which we highlight that our technique can effectively find symmetries and in turn achieve substantial reduction.

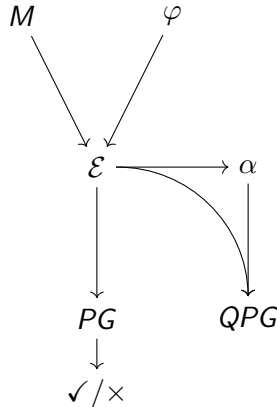
Keywords: Model checking · Parameterised Boolean equation systems · Symmetry reduction · Control flow · Parity games

1 Introduction

Within the area of model checking, symmetry reduction is a technique that aims to combat the well-known state space explosion problem. To reduce the effort spent on answering the model checking problem, symmetry reduction merges symmetric states of a system. Assume that states are represented as vectors of values. Then two states are symmetric if there exists a mapping that rearranges the state vectors into one another. During solving, a reduced state space is generated that takes into account the symmetries between states. This reduced state space is sufficient to answer the model checking problem fully. In turn, the model checking problem can be more efficiently answered.

Model checking tools like CUDD [1] and mCH2 [2] use parameterised Boolean equation systems (PBESs) [3] to encode the μ -calculus model checking problem. PBESs are systems of fixpoint equations ranging over predicate formulas. In mCH2, a PBES is constructed from a system specification and a μ -calculus formula to be verified. Solving a PBES and obtaining the answer to the model checking problem is achieved by converting it to a parity game [4]. This conversion results in state space explosion. Thus, the solving of the generated parity

* Artifact available at <https://doi.org/10.5281/zenodo.1933256>



Introduction

Control Flow-Based Symmetry Reduction for Parameterised Boolean Equation Systems*

Mounir Bartels^[1], Maurice Lavigne^[2], and Tim A.C. Willemse^[3]

Eindhoven University of Technology, Eindhoven, Netherlands
G.m.g.bartels@tue.nl, m.lavigne@tue.nl, t.a.c.willemse@tue.nl

Abstract. Symmetry reduction is a technique that reduces the state space explosion problem for model checking. The main idea of symmetry reduction is to merge symmetric states. Two states are symmetric if they can be mapped onto each other such that their semantics is preserved. To carry out symmetry reduction, one first has to identify symmetries for the system at hand. In this work we introduce the concept of symmetries for systems of fixed point equations called parameterised Boolean equation systems. We propose a technique that extracts a symmetry group from the control flow found in such an equation system. We implemented a prototype tool with which we highlight that our technique can effectively find symmetries and in turn achieve substantial reduction.

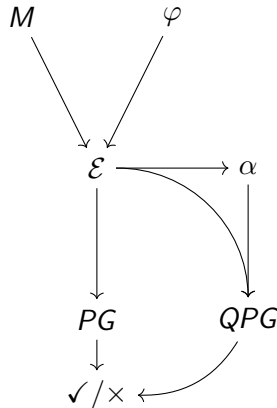
Keywords: Model checking · Parameterised Boolean equation systems · Symmetry reduction · Control flow · Parity games

1 Introduction

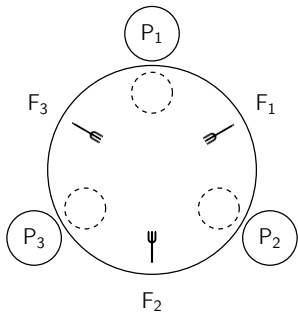
Within the area of model checking, symmetry reduction is a technique that aims to reduce the well-known state space explosion problem. To reduce the effort spent on solving the model checking problem, symmetry reduction merges symmetric states of a system. Assume that states are represented as vectors of values. Then two states are symmetric if there exists a mapping that rearranges the state vectors into one another. During solving, a reduced state space is generated that takes into account the symmetries between states. This reduced state space is sufficient to answer the model checking problem fully. In turn, the model checking problem can be more efficiently answered.

Model checking tools like CUDD [1] and mCHL2 [2] use parameterised Boolean equation systems (PBESs) [3] to encode the μ -calculus model checking problem. PBESs are systems of fixpoint equations ranging over predicate formulas. In mCHL2, a PBES is constructed from a system specification and a μ -calculus formula to be verified. Solving a PBES and obtaining the answer to the model checking problem is achieved by converting it to a parity game [4]. This conversion involves state space exploration. Then, the solving of the generated parity

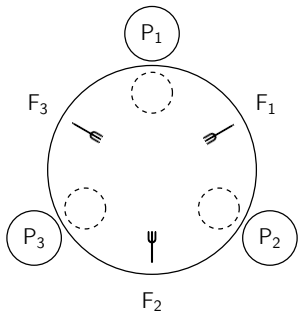
* Artifact available at <https://doi.org/10.5281/zenodo.1483226>



Introduction

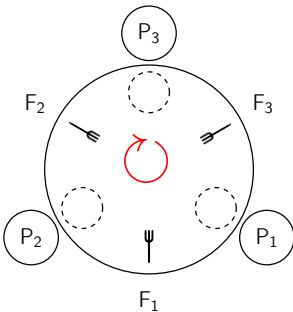
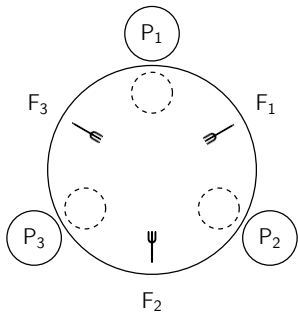


Introduction



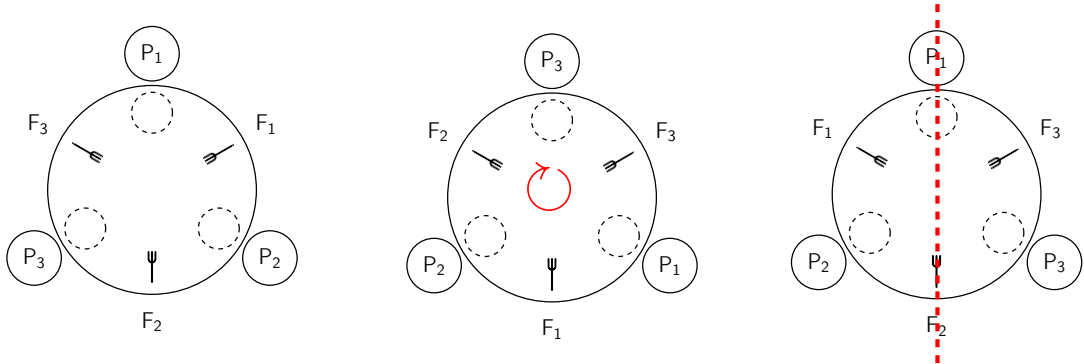
“Every philosopher picks up their left fork”

Introduction



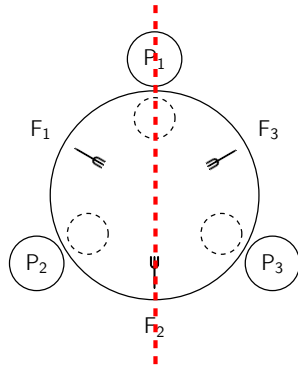
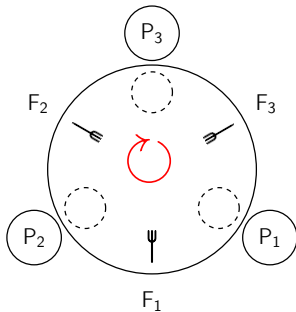
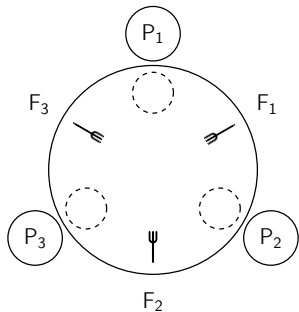
“Every philosopher picks up their left fork”

Introduction



“Every philosopher picks up their left fork”

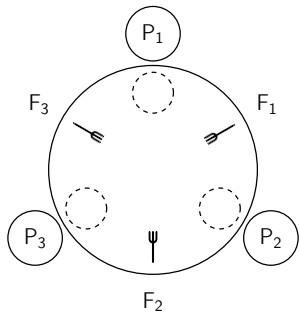
Introduction



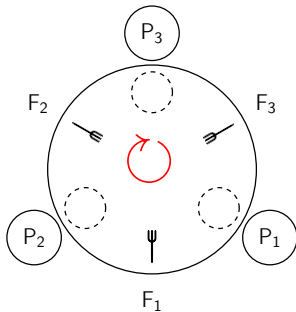
$\text{left_fork}(P_1) = F_1$

“Every philosopher picks up their left fork”

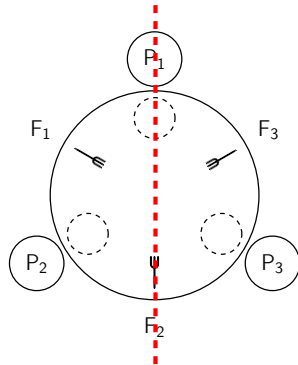
Introduction



$\text{left_fork}(P_1) = F_1$

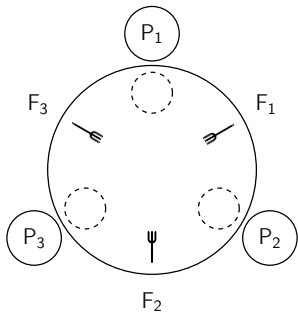


$\text{left_fork}(P_1) = F_1$

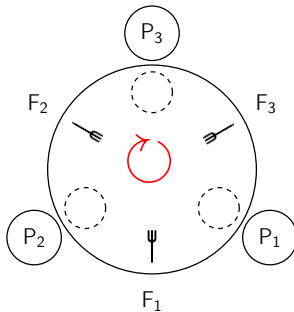


“Every philosopher picks up their left fork”

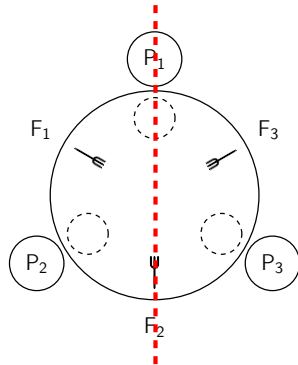
Introduction



$\text{left_fork}(P_1) = F_1$



$\text{left_fork}(P_1) = F_1$



$\text{left_fork}(P_1) = F_3$

“Every philosopher picks up their left fork”

Outline

Using colored graphs to extract symmetries for PBESs

Outline

Using **colored graphs** to extract symmetries for PBESs

Colored Graphs

Definition

A *vertex-edge-colored graph* is a tuple $G = (V, E, C)$ where V a set of vertices, $E \subseteq V \times V$ a set edges and $C: (V \cup E) \rightarrow K$ a coloring function for some set of colors K .

Definition

A *graph automorphism* is a bijection $h: V \rightarrow V$ such that

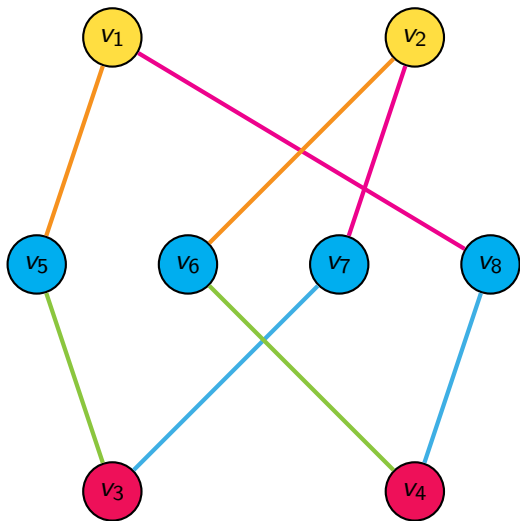
$$\begin{aligned} &\forall (v_1, v_2) \in E. (h(v_1), h(v_2)) \in E \\ &\forall v \in V. C(v) = C(h(v)) \quad \forall e \in E. C(e) = C(h(e)) \end{aligned}$$

We write $\text{Aut}(G)$ for the automorphism group of G

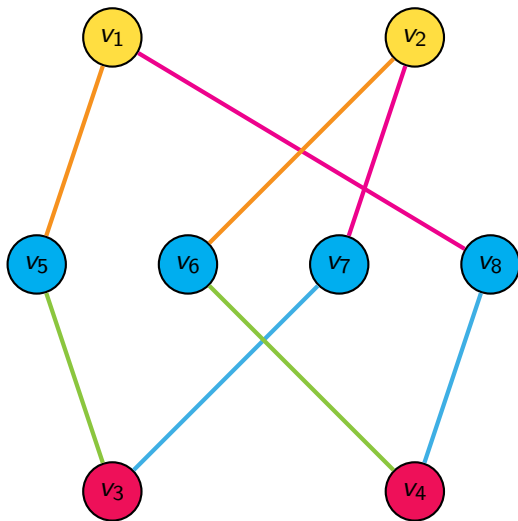
Colored Graphs



Colored Graphs



Colored Graphs



$$h: V \rightarrow V$$

$$\forall (v_1, v_2) \in E. (h(v_1), h(v_2)) \in E$$

$$\forall v \in V. C(v) = C(h(v))$$

$$\forall e \in E. C(e) = C(h(e))$$

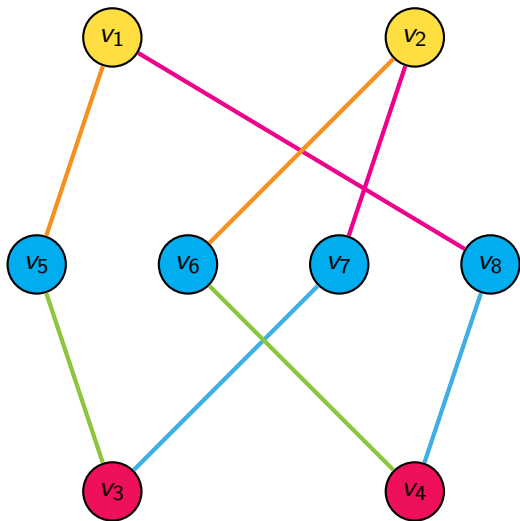
$$h(v_1) = v_2 \quad h(v_2) = v_1$$

$$h(v_3) = v_4 \quad h(v_4) = v_3$$

$$h(v_5) = v_6 \quad h(v_6) = v_5$$

$$h(v_7) = v_8 \quad h(v_8) = v_7$$

Colored Graphs



$$h: V \rightarrow V$$

$$\forall (v_1, v_2) \in E. (h(v_1), h(v_2)) \in E$$

$$\forall v \in V. C(v) = C(h(v))$$

$$\forall e \in E. C(e) = C(h(e))$$

$$h(v_1) = v_2 \quad h(v_2) = v_1$$

$$h(v_3) = v_4 \quad h(v_4) = v_3$$

$$h(v_5) = v_6 \quad h(v_6) = v_5$$

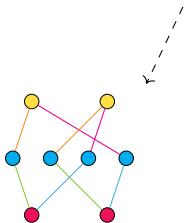
$$h(v_7) = v_8 \quad h(v_8) = v_7$$

$$h = (v_1 v_2)(v_3 v_4)(v_5 v_6)(v_7 v_8)$$

$$(h(v_1), h(v_5)) = (v_2, v_6)$$

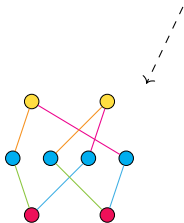
Outline

Using **colored graphs** to extract symmetries for PBESs



Outline

Using **colored graphs** to extract symmetries for **PBESs**



Parametrised Boolean Equation Systems

Definition

A Parametrised Boolean Equation System (PBES) is a sequence of fixpoint equations, inductively defined by the following grammar. Here $\sigma \in \{\nu, \mu\}$.

$$\mathcal{E} := \emptyset \mid (\sigma X(d : D) = \varphi)\mathcal{E} \quad \varphi := b \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid X(f) \mid \forall e : D. \varphi \mid \exists e : D. \varphi$$

Parametrised Boolean Equation Systems

Definition

A Parametrised Boolean Equation System (PBES) is a sequence of fixpoint equations, inductively defined by the following grammar. Here $\sigma \in \{\nu, \mu\}$.

$$\mathcal{E} := \emptyset \mid (\sigma X(d : D) = \varphi)\mathcal{E} \quad \varphi := b \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid X(f) \mid \forall e : D. \varphi \mid \exists e : D. \varphi$$

$$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$

Parametrised Boolean Equation Systems

Definition

A Parametrised Boolean Equation System (PBES) is a sequence of fixpoint equations, inductively defined by the following grammar. Here $\sigma \in \{\nu, \mu\}$.

$$\mathcal{E} := \emptyset \mid (\sigma X(d : D) = \varphi)\mathcal{E} \quad \varphi := b \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid X(f) \mid \forall e : D. \varphi \mid \exists e : D. \varphi$$

$$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$

We assume initial configuration $X(1, \text{tt}, 1, \text{tt})$.

Parametrised Boolean Equation Systems

$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$

Parametrised Boolean Equation Systems

$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$

$$X(1, \text{tt}, 1, \text{tt})$$

Parametrised Boolean Equation Systems

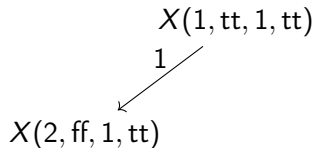
$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$



Parametrised Boolean Equation Systems

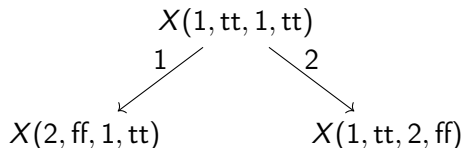
$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$



Parametrised Boolean Equation Systems

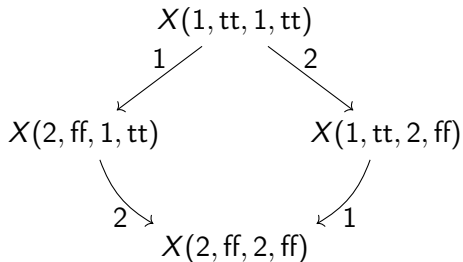
$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$



Parametrised Boolean Equation Systems

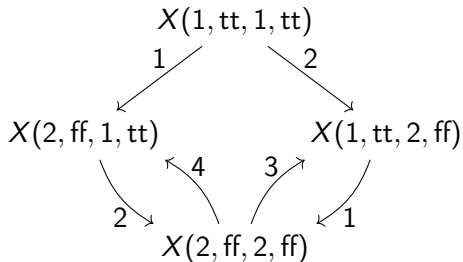
$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$



Control Flow Parameters

Definition

A parameter $c \in \text{par}(\mathcal{E})$ is a *control flow parameter* (CFP) if for every clause either $\text{src}(c)$ and $\text{tgt}(c)$ defined, or $\text{copy}(c)$ holds.

For CFPs we can deduce their unique value before and after every clause in $\mathcal{E}$.
Parameters that are not CFPs, are *data parameters* (DP). Notation: $\text{cfp}(\mathcal{E})$, $\text{dp}(\mathcal{E})$.

Control Flow Parameters

Definition

A parameter $c \in \text{par}(\mathcal{E})$ is a *control flow parameter* (CFP) if for every clause either $\text{src}(c)$ and $\text{tgt}(c)$ defined, or $\text{copy}(c)$ holds.

For CFPs we can deduce their unique value before and after every clause in $\mathcal{E}$.
Parameters that are not CFPs, are *data parameters* (DP). Notation: $\text{cfp}(\mathcal{E})$, $\text{dp}(\mathcal{E})$.

$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$

$$\begin{aligned} & (c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) & (1) & \text{cfp}(\mathcal{E}) = \{c_1, c_2\} \\ & \wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) & \text{dp}(\mathcal{E}) = \{d_1, d_2\} \\ & \wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) & (3) & \\ & \wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) & (4) & \end{aligned}$$

Control Flow Parameters

Definition

A data parameter $d \in \text{dp}(\mathcal{E})$ is:

- **Used for** a clause if it occurs in its guard
- **Used in** a clause if it used to update some other parameter
- **Changed by** a clause if its value is updated

$$d \in \text{uf}(j)$$

$$d \in \text{ui}(j)$$

$$d \in \text{cb}(j)$$

Control Flow Parameters

Definition

A data parameter $d \in \text{dp}(\mathcal{E})$ is:

- **Used for** a clause if it occurs in its guard
- **Used in** a clause if it used to update some other parameter
- **Changed by** a clause if its value is updated

$$d \in \text{uf}(j)$$

$$d \in \text{ui}(j)$$

$$d \in \text{cb}(j)$$

$$\begin{aligned} \mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) := \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4) \end{aligned}$$

	1	2	3	4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph

$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph



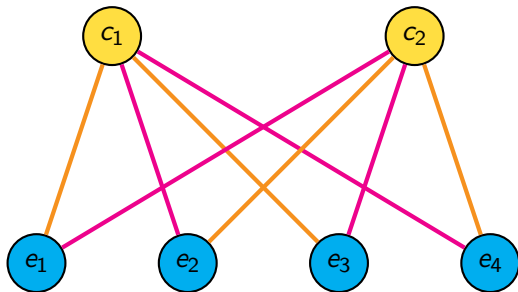
$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph



$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

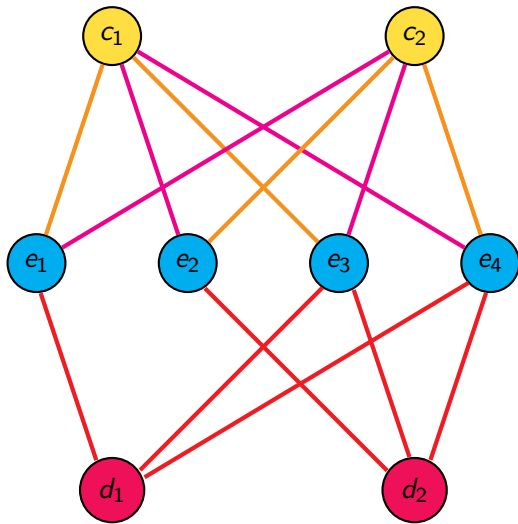
$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$



Coloring a Graph



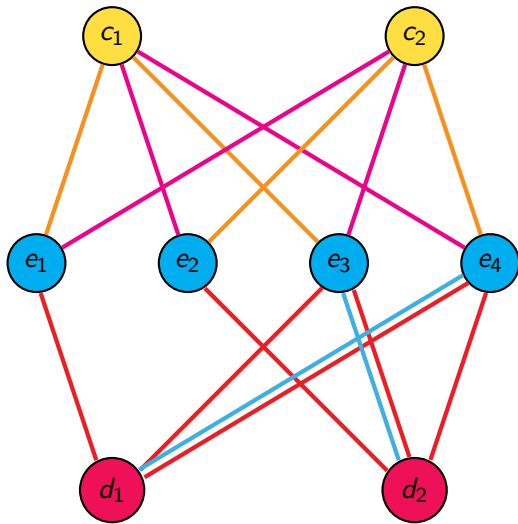
$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph



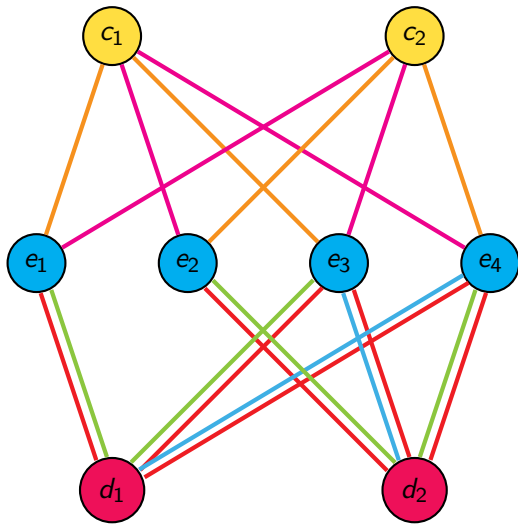
$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph



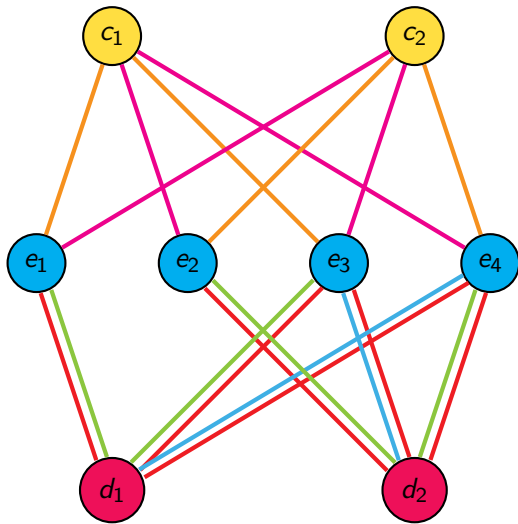
$$\text{cfp}(\mathcal{E}) = \{c_1, c_2\}$$

$$\text{dp}(\mathcal{E}) = \{d_1, d_2\}$$

src, tgt copy

	e_1	e_2	e_3	e_4
uf	$\{d_1\}$	$\{d_2\}$	$\{d_1, d_2\}$	$\{d_1, d_2\}$
ui	$\emptyset$	$\emptyset$	$\{d_2\}$	$\{d_1\}$
cb	$\{d_1\}$	$\{d_2\}$	$\{d_1\}$	$\{d_2\}$

Coloring a Graph



Restrict graph automorphisms to
domain of parameters :

$$h = (c_1 c_2)(d_1 d_2)(e_1 e_2)(e_3 e_4)$$

↓

$$\alpha = (c_1 c_2)(d_1 d_2)$$

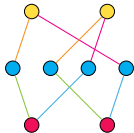
The permutation α now preserves
control flow information.

Outline

$$\begin{aligned}\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) &:= \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4)\end{aligned}$$



Using **colored graphs** to extract symmetries for **PBESs**

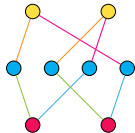


Outline

$$\begin{aligned}\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) &:= \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4)\end{aligned}$$



Using **colored graphs** to extract symmetries for PBESs



Symmetries

$$\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) :=$$

$$(c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2) \quad (1)$$

$$\wedge (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff}) \quad (2)$$

$$\wedge (c_1 = 2 \wedge d_1 = d_2) \Rightarrow X(1, \neg d_2, c_2, d_2) \quad (3)$$

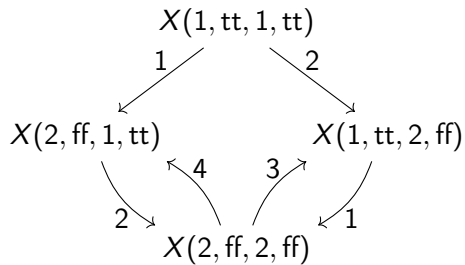
$$\wedge (c_2 = 2 \wedge d_2 = d_1) \Rightarrow X(c_1, d_1, 1, \neg d_1) \quad (4)$$

$$\alpha = (c_1 c_2)(d_1 d_2)$$

$$\alpha((c_1 = 1 \wedge d_1 = \text{tt}) \Rightarrow X(2, \text{ff}, c_2, d_2)) = (c_2 = 1 \wedge d_2 = \text{tt}) \Rightarrow X(c_1, d_1, 2, \text{ff})$$

$$“\alpha((1)) = (2)”$$

Symmetries



$$\alpha = (c_1 c_2)(d_1 d_2)$$

$$\alpha(X(2, ff, 1, tt)) = X(1, tt, 2, ff)$$

Symmetries

Definition

- A bijection $\alpha: \text{par}(\mathcal{E}) \rightarrow \text{par}(\mathcal{E})$ is a:
- Syntactic symmetry if $\mathcal{E} = \alpha(\mathcal{E})$
 - Semantic symmetry if $\llbracket \mathcal{E} \rrbracket = \llbracket \alpha(\mathcal{E}) \rrbracket$

$$\alpha = (c_1 c_2)(d_1 d_2)$$

$$\begin{aligned} \mathcal{E} = & \dots (c_1 = 1 \wedge d_1 = \text{tt}) \dots \\ & \dots (d_2 = \text{tt} \wedge c_2 = 1) \dots \end{aligned}$$

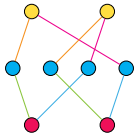
$\mapsto$

$$\begin{aligned} \alpha(\mathcal{E}) = & \dots (c_2 = 1 \wedge d_2 = \text{tt}) \dots \\ & \dots (d_1 = \text{tt} \wedge c_1 = 1) \dots \end{aligned}$$

Outline

$$\begin{aligned}\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) &:= \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4)\end{aligned}$$

Using **colored graphs** to extract symmetries for PBESs

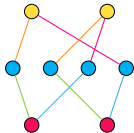


$$\alpha = (c_1 c_2)(d_1 d_2)$$

Outline

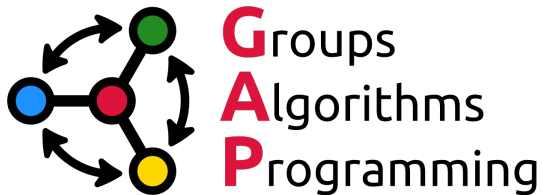
$$\begin{aligned}\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) &:= \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4)\end{aligned}$$

Using **colored graphs** to extract symmetries for PBESs



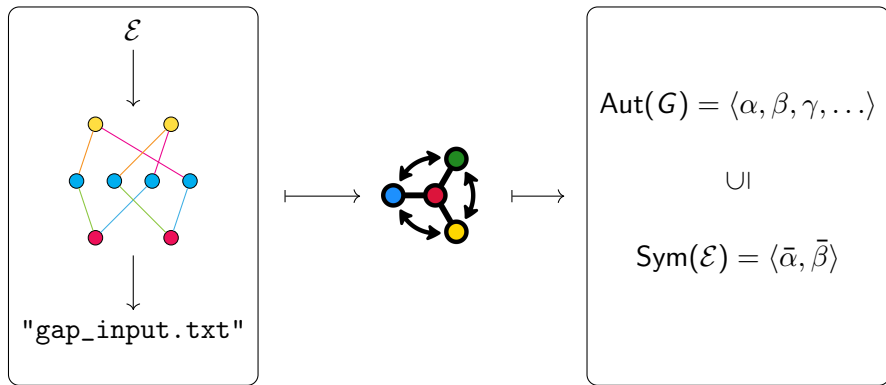
$$\alpha = (c_1 c_2)(d_1 d_2)$$

Extracting symmetries



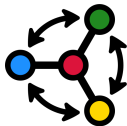
- “GAP is a system for computational discrete algebra, with particular emphasis on Computational Group Theory” –
<https://www.gap-system.org/>
- Package: GRaph Algorithms using PERmutation groups (GRAPE)
- Extract $\text{Aut}(G)$ for colored graphs

Extracting Symmetries



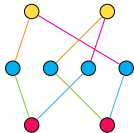
- Using CFGs this approach returns an over-approximation of syntactic symmetries
- Filtering done on group generators $\rightarrow$ compositional subgroup algorithm

Outline



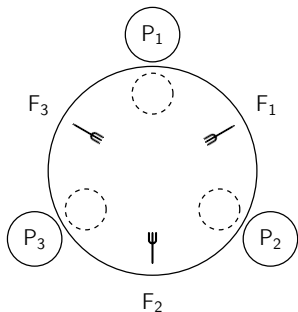
$$\begin{aligned}\mu X(c_1 : N, d_1 : B, c_2 : N, d_2 : B) &:= \\ (c_1 = 1 \wedge d_1 = \text{tt}) &\Rightarrow X(2, \text{ff}, c_2, d_2) & (1) \\ \wedge (c_2 = 1 \wedge d_2 = \text{tt}) &\Rightarrow X(c_1, d_1, 2, \text{ff}) & (2) \\ \wedge (c_1 = 2 \wedge d_1 = d_2) &\Rightarrow X(1, \neg d_2, c_2, d_2) & (3) \\ \wedge (c_2 = 2 \wedge d_2 = d_1) &\Rightarrow X(c_1, d_1, 1, \neg d_1) & (4)\end{aligned}$$

Using **colored graphs** to extract symmetries for PBESs



$$\alpha = (c_1 c_2)(d_1 d_2)$$

Dining Philosophers

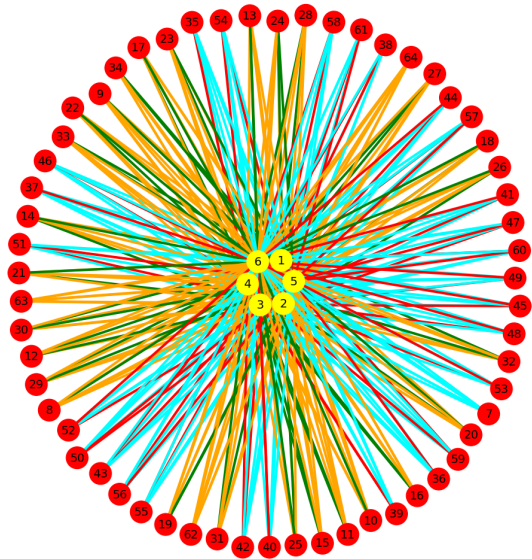


```
sort Phil = struct p1 | p2 | p3;
      Fork = struct f1 | f2 | f3;
map lf, rf: Phil -> Fork;
eqn lf(p1) = f1; lf(p2) = f2; lf(p3) = f3;
    rf(p1) = f3; rf(p2) = f1; rf(p3) = f2;
act get, put, up, down, lock, free: Fork;
  eat: Phil;

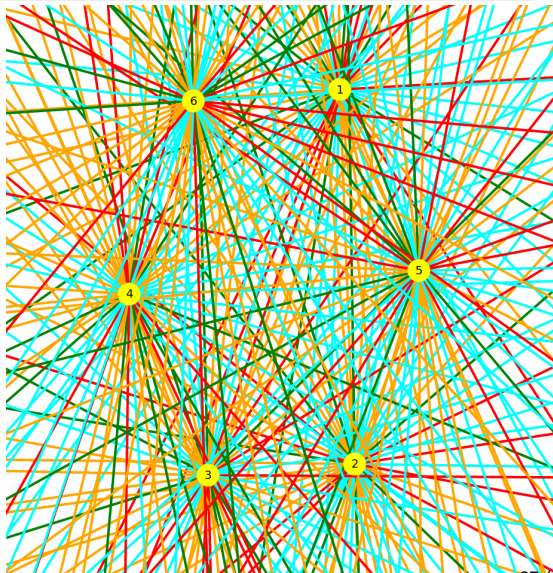
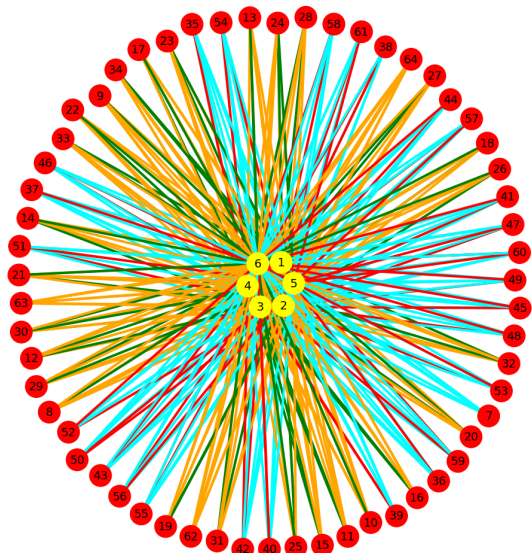
proc P_Phil(p: Phil) =
  (get(lf(p)).get(rf(p)) + get(rf(p)).get(lf(p))) . eat(p) .
  (put(lf(p)).put(rf(p)) + put(rf(p)).put(lf(p))) . P_Phil(p);
  P_Fork(f: Fork) = up(f) . down(f) . P_Fork(f);

init allow ( { lock, free, eat },
  comm( { get|up->lock, put|down->free },
    P_Fork(f1) || P_Fork(f2) || P_Fork(f3) ||
    P_Phil(p1) || P_Phil(p2) || P_Phil(p3) ));
```

Dining Philosophers



Dining Philosophers



Dining Philosophers

```
> ./gap
GAP 4.14.0 of 2024-12-05
https://www.gap-system.org
Architecture: x86_64-pc-linux-gnu-default64-kv9
Configuration: gmp 6.2.1, GASMAN, readline
Loading the library and packages ...
Packages: [...]
Try '??help' for help. See also '?copyright', '?cite' and '?authors'

gap> LoadPackage("grape");
-----
Loading GRAPE 4.9.3 (GRaph Algorithms using PERmutation groups)
by Leonard H. Soicher (https://webpace.maths.qmul.ac.uk/l.h.soicher/).
Homepage: https://gap-packages.github.io/grape
Report issues at https://github.com/gap-packages/grape/issues
-----
true

gap>
```

Dining Philosophers

```
gap> A := "412x412 adjacency matrix";
A := [[...],..., [...]]
gap> g := Graph( Group(()), [1..{}], OnPoints, function(x,y) return A[x][y]=1; end, true);
g := Graph (
  group := Group(()), isGraph := true,
  names := [ 1 .. 412 ], order := 412,
  representatives := [ 1, ..., 412],
  schreierVector := [ ... ]
)
gap> cols := "vertex partition";
cols = [[...],..., [...]]
gap> ag := AutGroupGraph(g, cols);
<permutation group with 40 generators>
gap> quit;

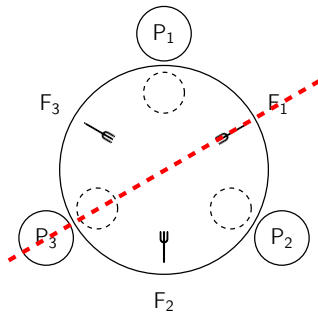
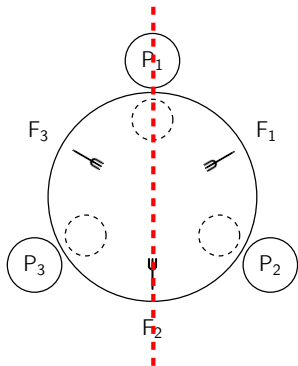
py parse_generators(GAP_ouptut)
[(2 3)(4 5), (1 3)(5 6)]
```

Dining Philosopher

$$\alpha_1 = (P_2P_3)(F_1F_3) \quad \alpha_2 = (P_1P_2)(F_2F_3)$$

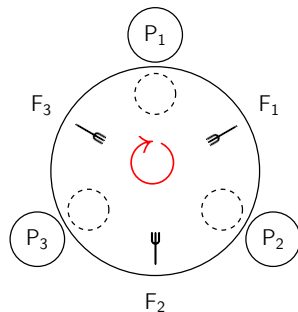
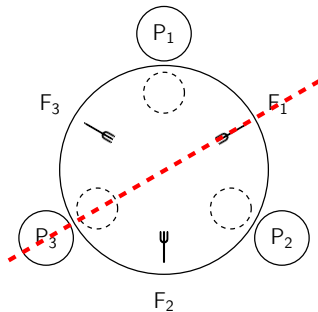
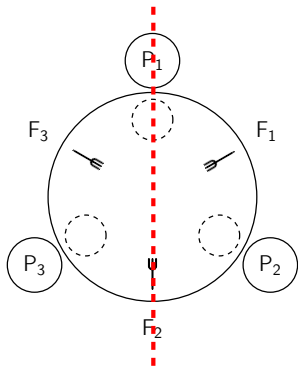
Dining Philosopher

$$\alpha_1 = (P_2 P_3)(F_1 F_3) \quad \alpha_2 = (P_1 P_2)(F_2 F_3)$$



Dining Philosopher

$$\alpha_1 = (P_2P_3)(F_1F_3) \quad \alpha_2 = (P_1P_2)(F_2F_3)$$



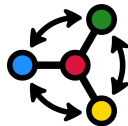
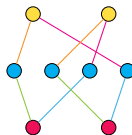
$$\alpha_1 \circ \alpha_2 = (P_1P_2P_3)(F_1F_2F_3)$$

Concluding

Using **colored graphs** to extract symmetries for PBESs

Further directions

- More/all syntax information in colored graph
- Data symmetries
- Add information about semantics



Thanks!

NetTCS Meeting - Wednesday 29 April 13.00-16.15 MF11/12

Network Types, Coalgebras, Semantics in NL

<https://nettcs.cs.ru.nl/index.html>

NetTCS is a joint initiative of research groups in theoretical computer science from University of Groningen, Leiden University, Open University, Radboud University, and University of Twente.

Subgroup Algorithm

Algorithm 1 Algorithm to find largest syntactic symmetry group of $\text{Aut}(G(\mathcal{E}))$

```
1:  $X :=$  generators of  $\text{Aut}(G(\mathcal{E}))$ 
2:  $H := \langle \{\alpha \in X \mid \alpha \text{ is a syntactic symmetry} \} \rangle$ 
3:  $U :=$  coset representatives of  $H$  in  $\text{Aut}(G(\mathcal{E}))$  except  $H$ 
4: while  $U \neq \emptyset$  do  $U := U \setminus \{\alpha\}$ 
5:   if  $\alpha$  is a syntactic symmetry then
6:      $H := \langle H \cup \{\alpha\} \rangle$ 
7:     if  $|\text{Aut}(G(\mathcal{E}))|/|H| < |U|$  then
8:        $U :=$  coset representatives of  $H$  in  $\text{Aut}(G(\mathcal{E}))$  except  $H$ 
9:     end if
10:  end if
11: end while
```
